Reflection Template

Use Case	UC-3
Use Case Name	Create Project
Requirement IDs	FR3, FR19, FR20, FR21, FR22, FR23
Matching Feature Comparison Sets	CS-6, CS-7, CS-8, CS-9
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Null safety, data classes/records, extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, the optional description made null safety really visible; in Kotlin it is simply String?, in Java there is no type-level difference between a required and optional field. the extension function was reused naturally from UC2. Data classes and records were used the same way as in UC1.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	5
Compactness of logic	3	5
Perceived implementation effort	4	5

Justifications

Readability:

In Kotlin the String? Makes it really visible that the description is optional in the type signature. In Java you have to check the validation annotations or the database column definition to understand that the description is optional. Other than that both implementations are basically the same.

Compactness of logic:

Kotlin reuses findByIdOrThrow from Extensions.kt. Java repeats orElseThrow inline again. The entity in Kotlin also handles the nullable description more naturally with a default value (= null), while Java just leaves it as a regular field.

Implementation effort:

Kotlin felt slightly easier since findByIdOrThrow was already there and String? required no extra thought. Java needed the same orElseThrow pattern again and the optional field had no explicit way to express it at the type level.

Observed structural differences

Kotlin expresses the optional description as String?, the nullability is part of the type. In Java description is just a normal String with no indication that it can be null at compile-time.

findByIdOrThrow from Extensions.kt is reused in ProjectService.kt without any changes. Java repeats the orElseThrow pattern inline again.

The Kotlin entity uses a default value (description: String? = null) making the optional field self-documenting. Java has no equivalent syntax.

DTOs are single-line records/data classes. Kotlin requires @field:NotBlank instead of just @NotBlank, this was not obvious at first and needed some research to figure out.

Final comparative summary

In this Use case null safety becomes really visible in a more meaningful way. Kotlin's String? clearly communicates that description is optional directly in the type, while Java offers no such signal. The extension function from UC-2 was reused without modification, which starts to show the long-term benefit of that abstraction and thus leads to less code already. Overall Kotlin remains more expressive, mainly due to explicit nullability and the reusable extension function.